

UNIT – 1

ASSIGNMENT – 1

WEB TECHNOLOGIES

SETHURAMAN S

2117230070147

B TECH AI&DS-C

INTERACTIVE TASK MANAGER

1. Introduction

The **Dynamic DOM Manipulation Website** is a simple interactive web application developed using **HTML, CSS, and JavaScript**. The main purpose of this project is to demonstrate how JavaScript can dynamically modify web page elements using **DOM (Document Object Model) manipulation**.

The website allows users to **add tasks, remove tasks, mark tasks as completed, and track progress visually**. It also includes modern UI features such as **animated backgrounds, hover effects, and dark mode** to improve the user experience.

This project helps learners understand how front-end technologies work together to create interactive web applications.

2. Objectives

The main objectives of this project are:

- To understand **DOM manipulation using JavaScript**
- To dynamically **add and remove HTML elements**
- To update webpage content without reloading the page
- To create an **interactive task management system**
- To implement **animations and responsive UI design**
- To track user progress using a **progress bar**

3. Technologies Used

1. HTML (HyperText Markup Language)

HTML is used to create the **structure of the website**, including sections such as the header, input fields, buttons, task list, and footer.

2. CSS (Cascading Style Sheets)

CSS is used to style the website and improve the visual appearance. Features include:

- Animated gradient background
- Button hover effects
- Task card design
- Progress bar styling
- Dark mode theme
- Smooth animations

3. JavaScript

JavaScript is used to add **dynamic functionality** to the website. It controls how users interact with the page, such as adding tasks, deleting tasks, toggling completion status, and updating the progress bar.

4. System Features

1. Add Task

Users can type a task into the input field and click the **Add Task** button. JavaScript dynamically creates a new task item and adds it to the list.

2. Delete Task

Each task contains a **Delete button** which removes the task from the list with a small animation effect.

3. Mark Task as Completed

Users can mark tasks as completed using a **checkbox**. Completed tasks are shown with a **strike-through effect**.

4. Remove All Tasks

The **Remove All button** clears the entire task list.

5. Change Theme

The **Change Theme button** toggles between **light mode** and **dark mode**.

6. Progress Bar

The progress bar visually shows the percentage of tasks that have been completed.

5. Website Structure

The project consists of three main files:

1. HTML File

The HTML file defines the layout of the webpage including:

- Header section
- Task input controls
- Progress bar section
- Task list display
- Footer

2. CSS File

The CSS file handles the design and animations such as:

- Gradient animated background
- Button hover effects
- Task card shadows
- Progress bar animation
- Dark mode styling

3. JavaScript File

The JavaScript file contains functions that manipulate the DOM:

Function	Description
addTask()	Adds a new task to the list
deleteTask()	Deletes a specific task
toggleTask()	Marks a task as completed

Function	Description
removeAll()	Removes all tasks
changeTheme()	Switches between dark and light themes
updateProgress()	Updates the progress bar

6. Program

Html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Dynamic DOM Manipulation</title>
<link rel="stylesheet" href="style.css">
</head>
<body>
<header>
<h1>Dynamic DOM Manipulation Website</h1>
<p>Add, Remove and Style Elements using JavaScript</p>
</header>
<section class="controls">
<input type="text" id="taskInput" placeholder="Enter a task">
<button onclick="addTask()">Add Task</button>
<button onclick="removeAll()">Remove All</button>
<button onclick="changeTheme()">Change Theme</button>
```

```
</section>
<section class="progress-section">
<h3>Task Progress</h3>
<div class="progress-container">
<div id="progressBar"></div>
</div>
</section>
<section>
<ul id="taskList"></ul>
</section>
<footer>
<p>DOM Manipulation Demo Project</p>
</footer>
<script src="script.js"></script>
</body>
</html>
```

CSS

```
body{
font-family: Arial, sans-serif;
margin:0;
padding:0;
text-align:center;
color:#333;
/* Animated gradient background */
background: linear-gradient(270deg,#74ebd5,#acb6e5,#fbc2eb);
```

```
background-size:600% 600%;
animation: gradientMove 12s ease infinite;
transition:0.4s;
}

@keyframes gradientMove{
0%{background-position:0% 50%;}
50%{background-position:100% 50%;}
100%{background-position:0% 50%;}
}

header{
background:rgba(0,0,0,0.6);
color:white;
padding:20px;
backdrop-filter:blur(5px);
}

.controls{
margin:20px;
}

input{
padding:10px;
width:250px;
border-radius:5px;
border:none;
outline:none;
}

/* Button Effects */
```

```
button{
padding:10px 15px;
margin:5px;
border:none;
background:#007BFF;
color:white;
cursor:pointer;
border-radius:6px;
transition:0.3s;
}
button:hover{
transform:scale(1.08);
box-shadow:0 5px 15px rgba(0,0,0,0.3);
}
/* Task list */
ul{
list-style:none;
padding:0;
max-width:500px;
margin:auto;
}
li{
background:white;
padding:12px;
margin:10px;
border-radius:8px;
```



```
display:flex;
justify-content:space-between;
align-items:center;
box-shadow:0 4px 10px rgba(0,0,0,0.2);
/* Task entry animation */
animation:fadeIn 0.4s ease;
transition:0.3s;
}
li:hover{
transform:translateY(-3px);
box-shadow:0 8px 20px rgba(0,0,0,0.3);
}
/* Fade animation */
@keyframes fadeIn{
from{
opacity:0;
transform:translateY(-10px);
}
to{
opacity:1;
transform:translateY(0);
}
}
/* Delete button */
.delete-btn{
background:#ff4d4d;
```

```
padding:6px 10px;
border-radius:4px;
cursor:pointer;
}

/* Completed tasks */
.completed span{
text-decoration:line-through;
color:gray;
}

/* Progress bar */
.progress-section{
max-width:500px;
margin:20px auto;
text-align:left;
}

.progress-container{
width:100%;
height:20px;
background:#ddd;
border-radius:10px;
overflow:hidden;
}

#progressBar{
height:100%;
width:0%;
background:linear-gradient(90deg,#00ff9d,#00c3ff);
```

```
transition:0.5s;
box-shadow:0 0 10px rgba(0,255,200,0.8);
}
/* Dark mode */
.dark{
background:#121212;
color:white;
}
.dark li{
background:#333;
}
/* Footer */
footer{
margin-top:30px;
padding:15px;
background:rgba(0,0,0,0.7);
color:white;
}
```

Javascript

```
function addTask(){
let input=document.getElementById("taskInput");
let task=input.value.trim();
if(task===""){
alert("Please enter a task");
return;
```

```
}
```

```
let li=document.createElement("li");
```

```
li.innerHTML=`
```

```
<input type="checkbox" onclick="toggleTask(this)">
```

```
<span>${task}</span>
```

```
<button class="delete-btn" onclick="deleteTask(this)">Delete</button>
```

```
`;
```

```
document.getElementById("taskList").appendChild(li);
```

```
input.value="";
```

```
updateProgress();
```

```
}
```

```
function deleteTask(button){
```

```
let task = button.parentElement;
```

```
task.style.opacity="0";
```

```
task.style.transform="translateX(50px)";
```

```
setTimeout(()=>{
```

```
task.remove();
```

```
updateProgress();
```

```
},300);
```

```
}
```

```
function toggleTask(checkbox){
```

```
let li=checkbox.parentElement;
```

```
li.classList.toggle("completed");
```

```
updateProgress();
```

```
}
```

```

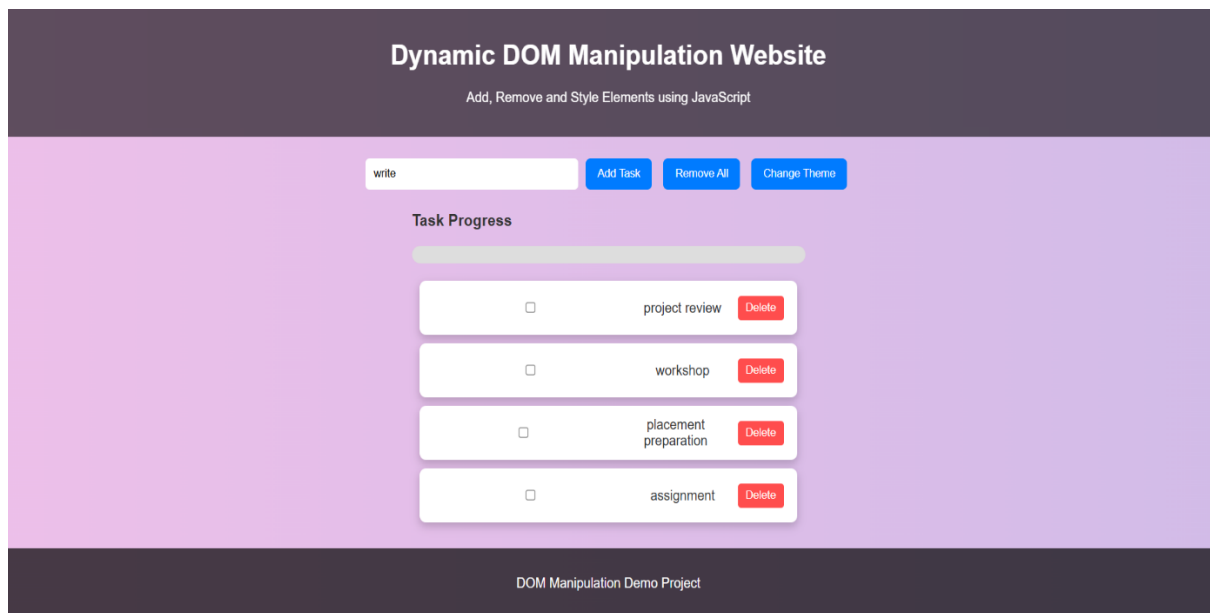
function removeAll(){
document.getElementById("taskList").innerHTML="";
updateProgress();
}

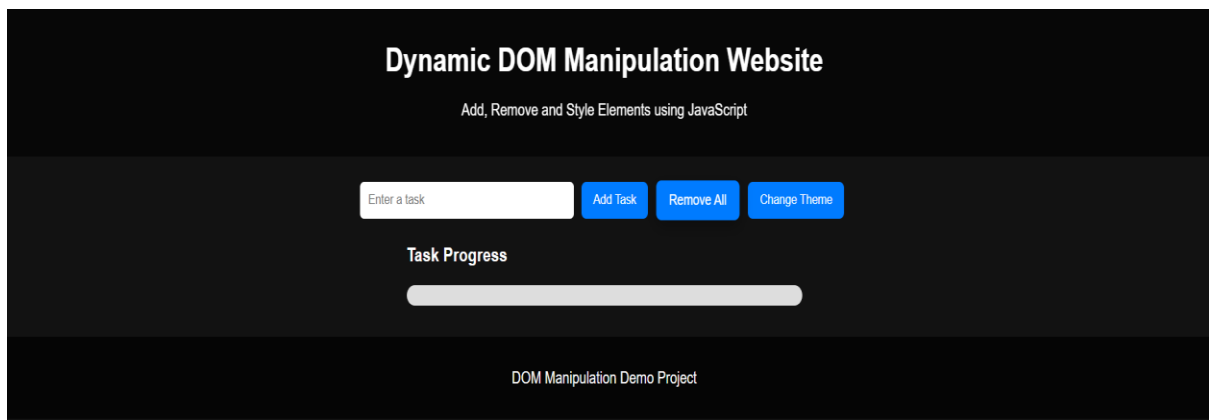
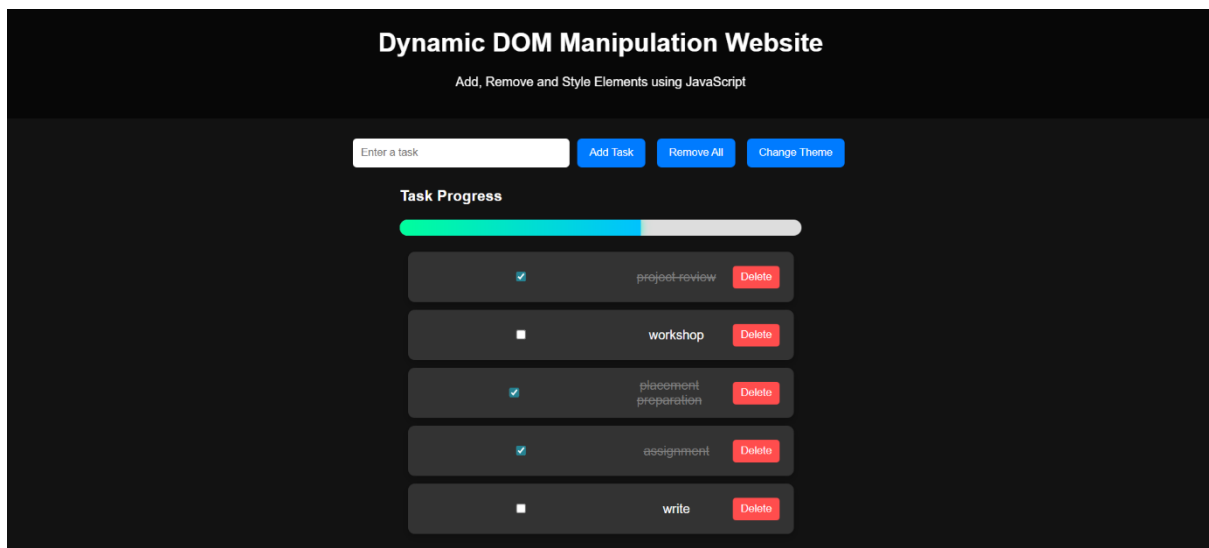
function changeTheme(){
document.body.classList.toggle("dark");
}

function updateProgress(){
let tasks=document.querySelectorAll("#taskList li");
let completed=document.querySelectorAll("#taskList li.completed");
let percent = tasks.length ? (completed.length / tasks.length) * 100 : 0;
document.getElementById("progressBar").style.width = percent + "%";
}

```

7.Output





8. Working of the System

1. The user enters a task in the input field.

2. When the **Add Task** button is clicked, JavaScript creates a new list element.
3. The task is added to the task list dynamically.
4. Users can mark tasks as completed using a checkbox.
5. The progress bar updates automatically based on completed tasks.
6. Tasks can be deleted individually or all tasks can be cleared at once.
7. Users can also switch between **light mode and dark mode**.

9. Advantages

- Easy to use and interactive
- Demonstrates practical **DOM manipulation**
- Includes modern UI animations
- Helps beginners understand **JavaScript event handling**
- Works directly in a web browser without installation

10. Limitations

- Tasks are not stored permanently
- Data will be lost if the page is refreshed
- No user authentication or database integration

11. Future Enhancements

The project can be improved by adding:

- **Local Storage** to save tasks permanently
- **Task editing feature**
- **Task categories or priority levels**
- **Responsive mobile design**

12. Conclusion

The **Dynamic DOM Manipulation Website** successfully demonstrates how JavaScript can dynamically control webpage elements. The project combines **HTML, CSS, and JavaScript** to create an interactive task management system with modern visual effects.

It is a useful beginner-level project that helps in understanding **web development concepts such as DOM manipulation, event handling, and dynamic UI updates**.